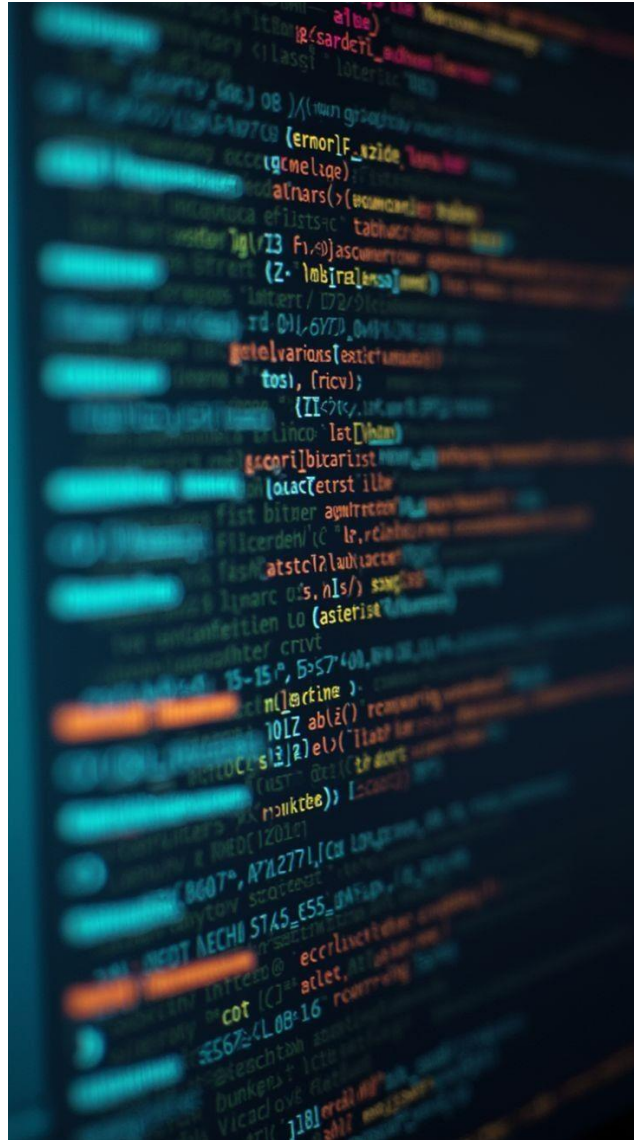


Projet EasySave

Livrable 01



Groupe 02 :

- MESSADI Mahmoud
- RECHAM Wissam
- LOUZAZNA Rayane
- BOUYACOUB Rayan

Table des matières

Introduction :	3
1. Analyse de besoin.....	4
1.1. Description du besoin global	4
1.2. Objectifs	4
1.3. Gestion des exécutions	4
2. Choix d'architecture	5
3. Diagramme UML.....	6
3.1. Interactions entre les Acteurs Externes	6
3.2. Organisation et structure statique du système	7
3.3. Interactions des Objets au Fil du Temps.....	10
3.4. Flux de Travail et Processus dans le Système.....	13
Conclusion	16

Tables des figures

Figure 1- Diagramme Use Case	6
Figure 2 - Diagramme de classe.....	8
Figure 3 - Diagramme de séquence.....	11
Figure 4 - Diagramme d'activité	15

Introduction

Dans le cadre de son développement, l'entreprise ProSoft souhaite enrichir sa suite logicielle avec un outil de sauvegarde fiable, performant et évolutif nommé **EasySave**. Ce logiciel a pour vocation de permettre aux utilisateurs de gérer facilement des opérations de sauvegarde de fichiers, tout en garantissant la traçabilité des actions et la sécurité des données.

Le projet EasySave s'inscrit dans une logique industrielle où la qualité du logiciel, sa maintenabilité et sa capacité d'évolution sont des éléments essentiels. En effet, le produit est destiné à être déployé chez différents clients, avec des environnements variés (disques locaux, supports externes, réseaux). Il est donc nécessaire de concevoir une solution robuste, modulaire et adaptable.

Le développement du projet est structuré en plusieurs versions successives. La version 1.0, correspondant à ce premier livrable, constitue une base fonctionnelle reposant sur une application console. Elle permet de valider les principaux mécanismes du logiciel, notamment la gestion des travaux de sauvegarde, leur exécution, ainsi que la génération de fichiers de suivi (logs et états). Les versions ultérieures introduiront des évolutions, notamment une interface graphique et des fonctionnalités avancées.

Ce Livrable 1 a pour objectif principal de poser les fondations du système à travers une phase d'analyse et de conception. Il ne s'agit pas encore de fournir une implémentation complète, mais de modéliser le comportement et la structure du logiciel à l'aide de diagrammes UML. Cette approche permet de formaliser les besoins, d'anticiper les interactions entre les composants et de garantir une cohérence globale de l'architecture.

Dans ce document, nous présentons tout d'abord l'analyse des besoins fonctionnels et non fonctionnels du système. Nous détaillons ensuite les choix d'architecture retenus, en mettant en avant les principes de modularité et de séparation des responsabilités. Enfin, nous exposons les différents diagrammes UML réalisés (cas d'utilisation, classes, séquence et activité), qui illustrent le fonctionnement global du logiciel EasySave.

L'ensemble de ce travail vise à fournir une base solide, claire et structurée, facilitant la poursuite du développement dans les versions futures tout en assurant une bonne compréhension du système par l'ensemble des intervenants du projet.

1. Analyse de besoin

1.1. Description du besoin global

Le système doit permettre à un utilisateur de définir des travaux de sauvegarde, chacun représentant une opération de copie entre un répertoire source et un répertoire cible. Chaque travail est identifié par un nom unique et peut être exécuté individuellement ou en groupe.

Lors de l'exécution d'un travail, le logiciel doit parcourir l'ensemble des fichiers du répertoire source, y compris les sous-dossiers, afin d'assurer une sauvegarde complète de la structure. Chaque fichier doit être copié vers la destination correspondante en conservant l'arborescence.

Le système doit également assurer un suivi précis de l'exécution des sauvegardes. Cela inclut la mise à jour d'un état décrivant l'avancement du travail (nombre de fichiers traités, fichiers restants, progression, taille totale, etc.) ainsi que l'écriture d'un journal détaillé des actions réalisées.

1.2. Objectifs

Le logiciel EasySave a pour objectif principal de fournir une solution de sauvegarde simple, fiable et extensible permettant aux utilisateurs de sécuriser leurs données à travers des opérations de copie entre répertoires sources et destinations.

Dans un contexte professionnel, ce type de logiciel doit répondre à des exigences de stabilité, de traçabilité et de flexibilité afin de s'adapter à différents environnements clients (postes locaux, serveurs, disques externes ou réseaux).

Les objectifs principaux du système sont les suivants :

- Permettre la création et la gestion de travaux de sauvegarde personnalisés
- Assurer l'exécution fiable des sauvegardes de fichiers et dossiers
- Garantir le suivi en temps réel de l'avancement des opérations
- Fournir une trace complète des actions effectuées pour audit et maintenance
- Offrir une interface simple en mode console pour l'utilisateur
- Permettre une exécution automatisée via ligne de commande
- Préparer une architecture évolutive pour les futures versions (notamment interface graphique)

1.3. Gestion des exécutions

Le logiciel doit supporter deux modes principaux d'utilisation :

- Un mode interactif basé sur un menu permettant à l'utilisateur de choisir les actions à effectuer (ajout, suppression, exécution des travaux, etc.)
- Un mode automatisé via ligne de commande permettant l'exécution directe de certains travaux sans interaction utilisateur

Cette double approche permet de répondre à différents contextes d'utilisation, notamment l'utilisation manuelle ou l'intégration dans des scripts d'automatisation.

2.Choix d'architecture

L'architecture logicielle correspond à la manière dont un système est structuré et organisé en différents composants, ainsi qu'à la façon dont ces composants interagissent entre eux. Elle définit les grandes lignes du fonctionnement interne de l'application, indépendamment des détails d'implémentation.

Dans le cadre du projet EasySave, une architecture claire et structurée est essentielle afin de garantir un logiciel maintenable, évolutif et facile à comprendre. En effet, une mauvaise organisation du code peut rapidement entraîner des difficultés lors de l'ajout de nouvelles fonctionnalités, de la correction de bugs ou de la réutilisation du système dans d'autres contextes.

L'architecture joue donc un rôle fondamental dans la qualité globale du logiciel. Elle permet notamment de :

- Séparer les responsabilités entre les différentes parties du système,
- Améliorer la lisibilité et la compréhension du code,
- Faciliter la maintenance et les évolutions futures,
- Réduire les dépendances entre les composants,
- Favoriser la réutilisabilité des modules.

Pour répondre à ces objectifs, le projet EasySave adopte une architecture inspirée du modèle **MVVM (Model – ViewModel)**, combinée à une approche en couches (Layered Architecture).

Cette organisation repose sur une séparation claire des rôles :

- Les Models représentent les données du système (BackupJob, BackupType).
- Les ViewModels contiennent la logique métier et assurent la coordination entre les données et les services.
- Les Services gèrent les traitements spécifiques comme la sauvegarde, la configuration ou la gestion de la langue.
- L'interface **Console** permet l'interaction avec l'utilisateur.

Cette séparation permet d'éviter un code monolithique difficile à maintenir et favorise une évolution progressive du logiciel. Par exemple, l'ajout d'une interface graphique dans une version future (v2.0) pourra être réalisé sans modifier profondément la logique métier existante.

De plus, le projet intègre une bibliothèque externe **EasyLog.dll**, dédiée à la gestion des logs et de l'état des sauvegardes. Cette séparation en module indépendant améliore la réutilisabilité du système et garantit une meilleure organisation du code.

3. Diagramme UML

Afin de formaliser les besoins et de structurer la conception du logiciel *EasySave v1.0*, plusieurs diagrammes UML ont été réalisés. Ces diagrammes permettent de représenter le système sous différents angles : les interactions entre l'utilisateur et le logiciel, l'architecture interne, ainsi que le déroulement des principaux processus.

L'objectif de cette modélisation est d'assurer une compréhension claire du système, de faciliter la communication entre les développeurs et de garantir une architecture cohérente et évolutive.

3.1. Interactions entre les Acteurs Externes

Le diagramme de cas d'utilisation représente les fonctionnalités accessibles à l'utilisateur ainsi que les interactions principales avec le système.

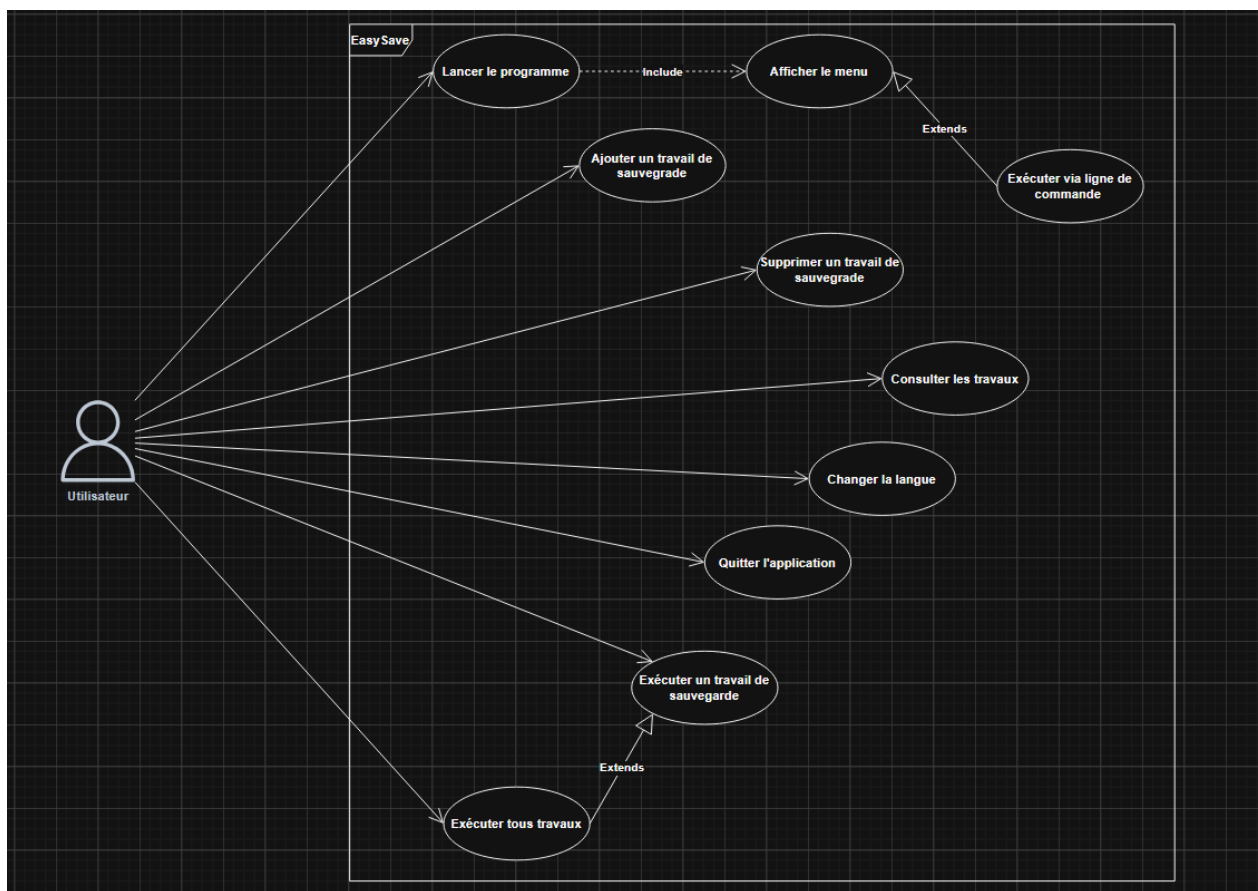


Figure 1- Diagramme Use Case

Il met en évidence :

- Le lancement de l'application
- La gestion des travaux de sauvegarde (ajout, suppression, consultation)
- L'exécution des sauvegardes (unitaire ou globale)
- Le changement de langue
- L'exécution via ligne de commande
- La sortie de l'application

Ce diagramme distingue également deux modes d'utilisation :

- Mode interactif (menu utilisateur)
- Mode ligne de commande (CLI)

3.2. Organisation et structure statique du système

Le diagramme de classes du projet EasySave présente l'architecture logicielle globale du système. Il suit une approche modulaire basée sur la séparation des responsabilités afin d'assurer la maintenabilité, l'évolutivité et la réutilisabilité du code.

Le système est organisé en plusieurs packages logiques :

- EasyLog (gestion des logs et états)
- Models (données métier)
- ViewModels (logique applicative)
- Services (traitements métier)
- Console (interface utilisateur)

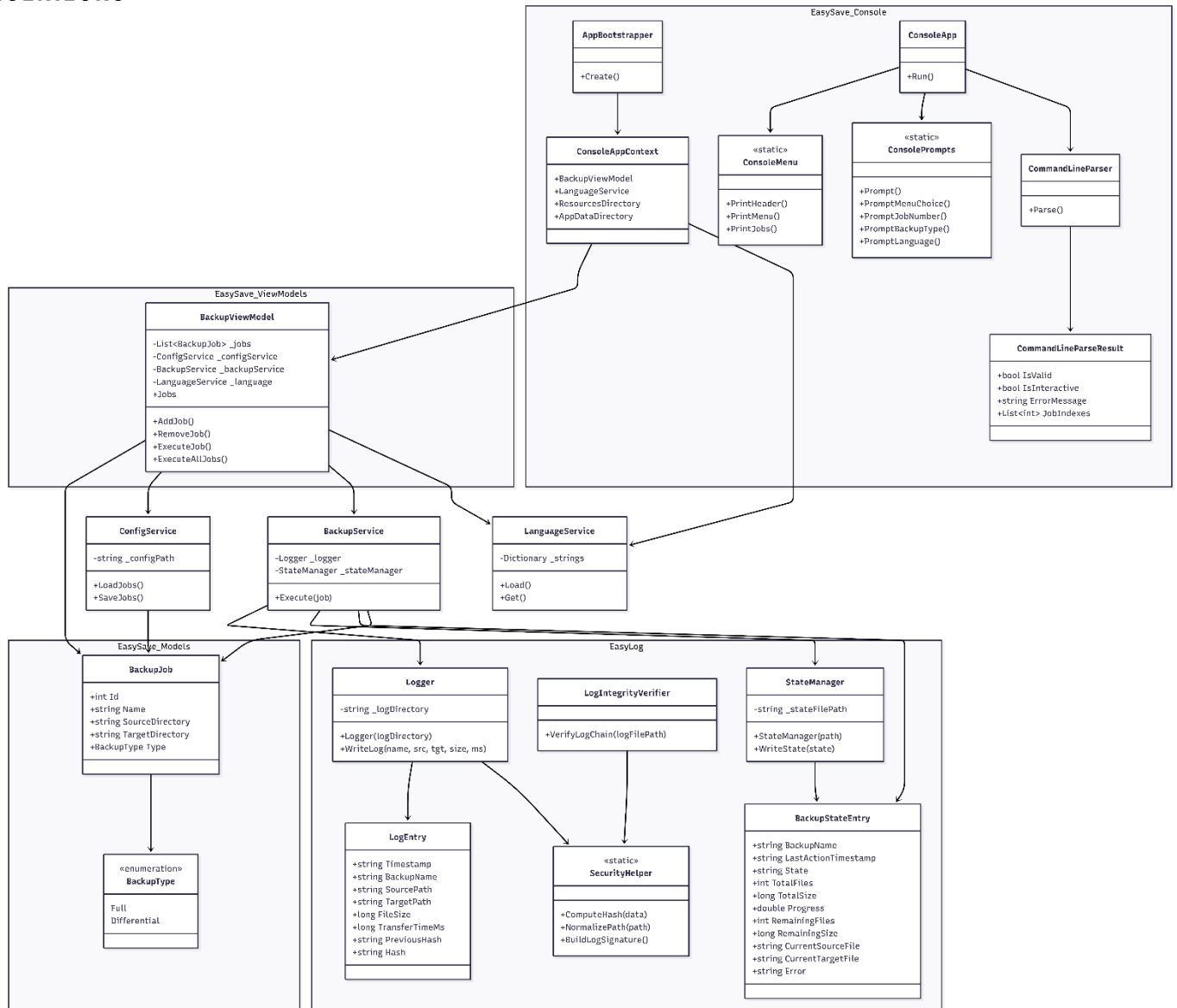


Figure 2 - Diagramme de classe

Package EasyLog (gestion des logs et de l'état)

Ce package contient les classes responsables du suivi des sauvegardes et de la traçabilité.

LogEntry : représente une ligne de log. Elle contient les informations d'une action de sauvegarde (fichier source, destination, taille, temps de transfert, etc.).

BackupStateEntry : représente l'état en temps réel d'un travail de sauvegarde (progression, fichiers restants, état actuel, erreurs).

Logger : classe principale qui écrit les logs dans des fichiers.

StateManager : enregistre et met à jour l'état des sauvegardes.

SecurityHelper (statique) : fournit des fonctions utilitaires liées à la sécurité comme le calcul de hash.

LogIntegrityVerifier : vérifie l'intégrité des fichiers de log.

- Ce package est indépendant et peut être réutilisé (DLL EasyLog).

Package Models (données métier)

Ce package contient les objets principaux du domaine.

BackupJob : représente un travail de sauvegarde (nom, dossier source, dossier cible, type).

BackupType : enum qui définit le type de sauvegarde :

Full (complète)

Differential (différentielle)

- Ce sont les données de base utilisées par tout le système.

Package ViewModels (logique principale)

BackupViewModel : c'est le cœur de l'application.

- Il gère la liste des jobs
- Il permet d'ajouter, supprimer et exécuter des sauvegardes
- Il fait le lien entre les services et les données

- Il dépend de plusieurs services :

ConfigService (chargement/sauvegarde des jobs)

BackupService (exécution des sauvegardes)

LanguageService (gestion des langues)

Services (logique métier)

ConfigService : gère la lecture et l'écriture des configurations (liste des jobs).

BackupService : exécute réellement les sauvegardes et utilise le Logger + StateManager.

LanguageService : gère les textes multilingues (FR / EN).

- Ces services contiennent la logique principale du système.

Package Console (interface utilisateur)

Ce package gère l'interaction avec l'utilisateur via le terminal.

AppBootstrapper : initialise l'application.

ConsoleAppContext : contient les dépendances globales.

ConsoleApp : lance l'application console.

ConsoleMenu : affiche le menu utilisateur.

ConsolePrompts : gère les entrées utilisateur.

CommandLineParser : analyse les arguments de la ligne de commande.

CommandLineParseResult : contient le résultat du parsing.

- Ce package est la couche d'interface entre l'utilisateur et le système.

Relations principales

BackupViewModel est le centre du système (il utilise les services et les jobs)

BackupService utilise :

- Logger
- StateManager
- BackupJob
- ConfigService travaille avec les BackupJobs
- ConsoleApp communique avec le ViewModel via le contexte
- EasyLog est un module indépendant utilisé pour logs et état

3.3. Interactions des Objets au Fil du Temps

Le diagramme de séquence illustre le déroulement dynamique d'une exécution de sauvegarde dans le système EasySave. Il met en évidence les interactions entre les différents composants du système ainsi que les deux modes possibles d'utilisation : le mode ligne de commande (CLI) et le mode interactif.

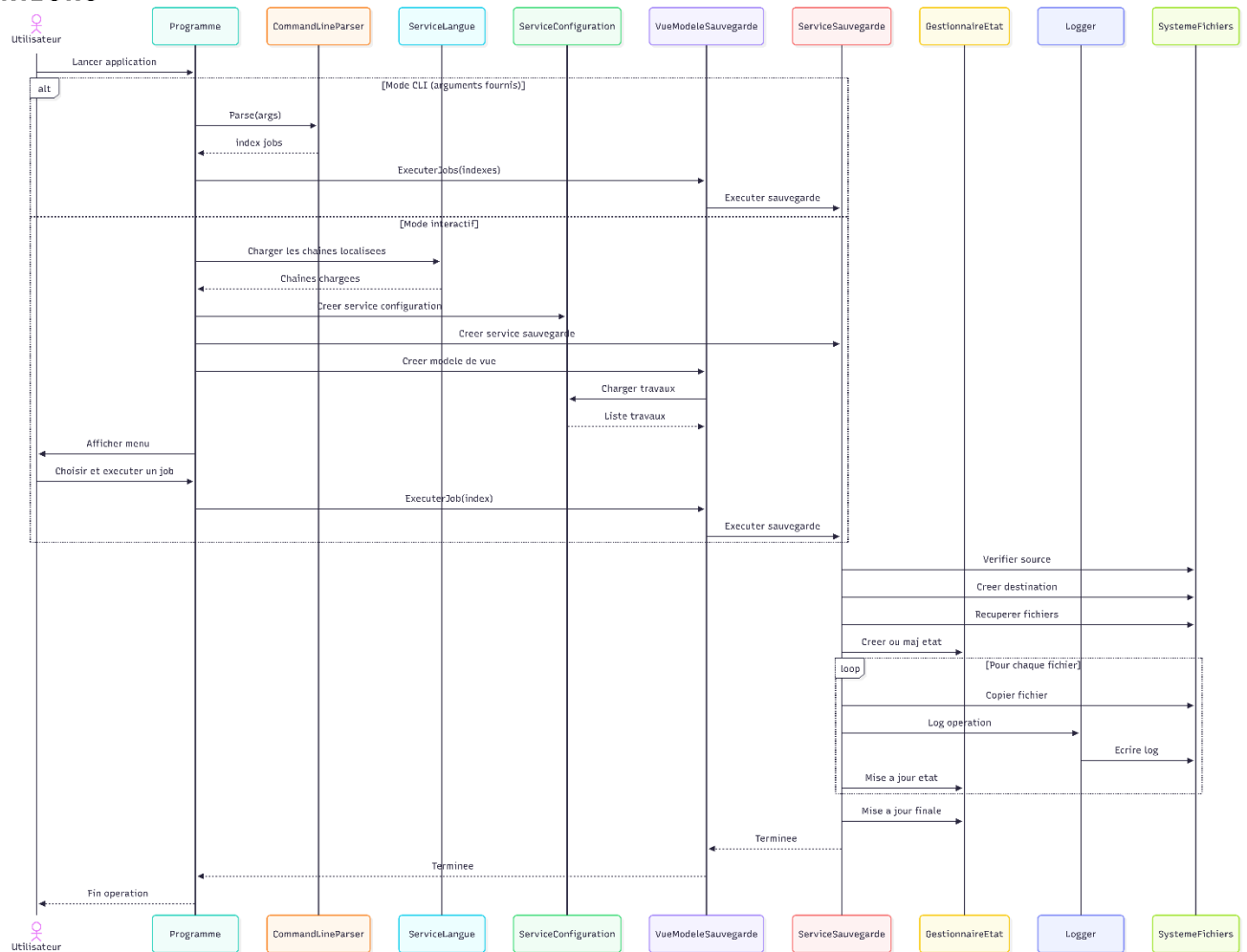


Figure 3 - Diagramme de séquence

Démarrage de l'application

Le processus commence lorsque l'**utilisateur lance l'application**. Le programme est alors initialisé et doit déterminer le mode d'exécution :

- Soit via des **arguments en ligne de commande**
- Soit via une **interaction utilisateur classique (menu)**

Choix du mode d'exécution (alt)

Le diagramme utilise un bloc alt pour représenter deux scénarios alternatifs :

- Mode CLI (automatique)

Si des arguments sont fournis :

Le programme appelle le **CommandLineParser** pour analyser les arguments.

Les indices des travaux à exécuter sont extraits.

Le **ViewModel** lance directement l'exécution des sauvegardes demandées via le **ServiceSauvegarde**.

Ce mode permet une exécution rapide sans interaction utilisateur.

- Mode interactif

Si aucun argument n'est fourni :

Le programme charge les **chaînes de langue** via le ServiceLangue.

Il initialise les services nécessaires :

- ServiceConfiguration
- ServiceSauvegarde
- VueModeleSauvegarde

Les travaux de sauvegarde sont chargés depuis la configuration.

Ensuite :

- Le menu est affiché à l'utilisateur.
- L'utilisateur sélectionne un travail à exécuter.
- Le ViewModel déclenche la sauvegarde via le service dédié.

Exécution de la sauvegarde

Une fois la sauvegarde lancée, le **ServiceSauvegarde** prend en charge le traitement principal :

- Vérification du répertoire source
- Création du répertoire de destination
- Récupération des fichiers à sauvegarder

Gestion de l'état et des logs

Pendant l'exécution :

- Le Gestionnaire d'état crée ou met à jour l'état du travail de sauvegarde
- Le Logger enregistre chaque action (copie de fichier, progression, etc.)
- Les opérations sont écrites en temps réel dans les fichiers de log

Traitement des fichiers (loop)

Le bloc loop représente le traitement répétitif :

Pour chaque fichier :

- Copie du fichier vers la destination
- Journalisation de l'opération
- Mise à jour de l'état d'avancement

Finalisation

Une fois tous les fichiers traités :

L'état final est mis à jour

Le service de sauvegarde informe le ViewModel de la fin de l'opération

Le programme notifie l'utilisateur que l'exécution est terminée

3.4. Flux de Travail et Processus dans le Système

Le diagramme d'activité représente le flux global d'exécution du logiciel EasySave. Il décrit de manière séquentielle et conditionnelle les différentes étapes suivies par le système depuis son lancement jusqu'à la fin d'une opération de sauvegarde.

Ce diagramme met en évidence deux modes principaux d'utilisation : le mode ligne de commande (CLI) et le mode interactif (menu utilisateur), ainsi que le processus commun d'exécution d'une sauvegarde.

Démarrage de l'application

Le processus commence par le lancement du programme. Le système initialise ensuite l'application et vérifie si des arguments ont été fournis en ligne de commande.

Une décision est prise :

- Si des arguments sont présents → mode CLI
- Sinon → mode interactif

Mode ligne de commande (CLI)

Dans ce mode, l'utilisateur exécute directement le programme avec des paramètres.

Le système effectue les étapes suivantes :

- Analyse des arguments fournis
- Identification des travaux de sauvegarde à exécuter
- Lancement automatique des sauvegardes sélectionnées
- Ce mode permet une exécution rapide sans interaction utilisateur.

Mode interactif (menu utilisateur)

Si aucun argument n'est fourni, l'application entre en mode interactif.

Le système :

- Charge la langue (FR/EN)

- Charge la configuration des travaux
- Affiche le menu principal

L'utilisateur peut ensuite choisir une action parmi :

- Ajouter un travail de sauvegarde
- Supprimer un travail
- Exécuter un travail
- Exécuter tous les travaux
- Quitter l'application

Lancement d'une sauvegarde

Qu'il s'agisse du mode CLI ou du mode interactif, l'exécution d'une sauvegarde suit un processus commun.

Traitement de la sauvegarde

Le système réalise les étapes suivantes :

- Initialisation de la sauvegarde
- Vérification de l'existence du répertoire source
- Création du répertoire cible si nécessaire
- Récupération de la liste des fichiers à copier

Ensuite, un traitement itératif est effectué pour chaque fichier :

- Copie du fichier vers la destination
- Journalisation de l'opération (log)
- Mise à jour de l'état d'avancement

Ce cycle se répète jusqu'à ce que tous les fichiers soient traités.

Fin de la sauvegarde

Une fois tous les fichiers copiés :

- Une mise à jour finale de l'état est effectuée
- Le processus de sauvegarde se termine

L'application peut ensuite revenir au menu ou se fermer selon le mode utilisé.

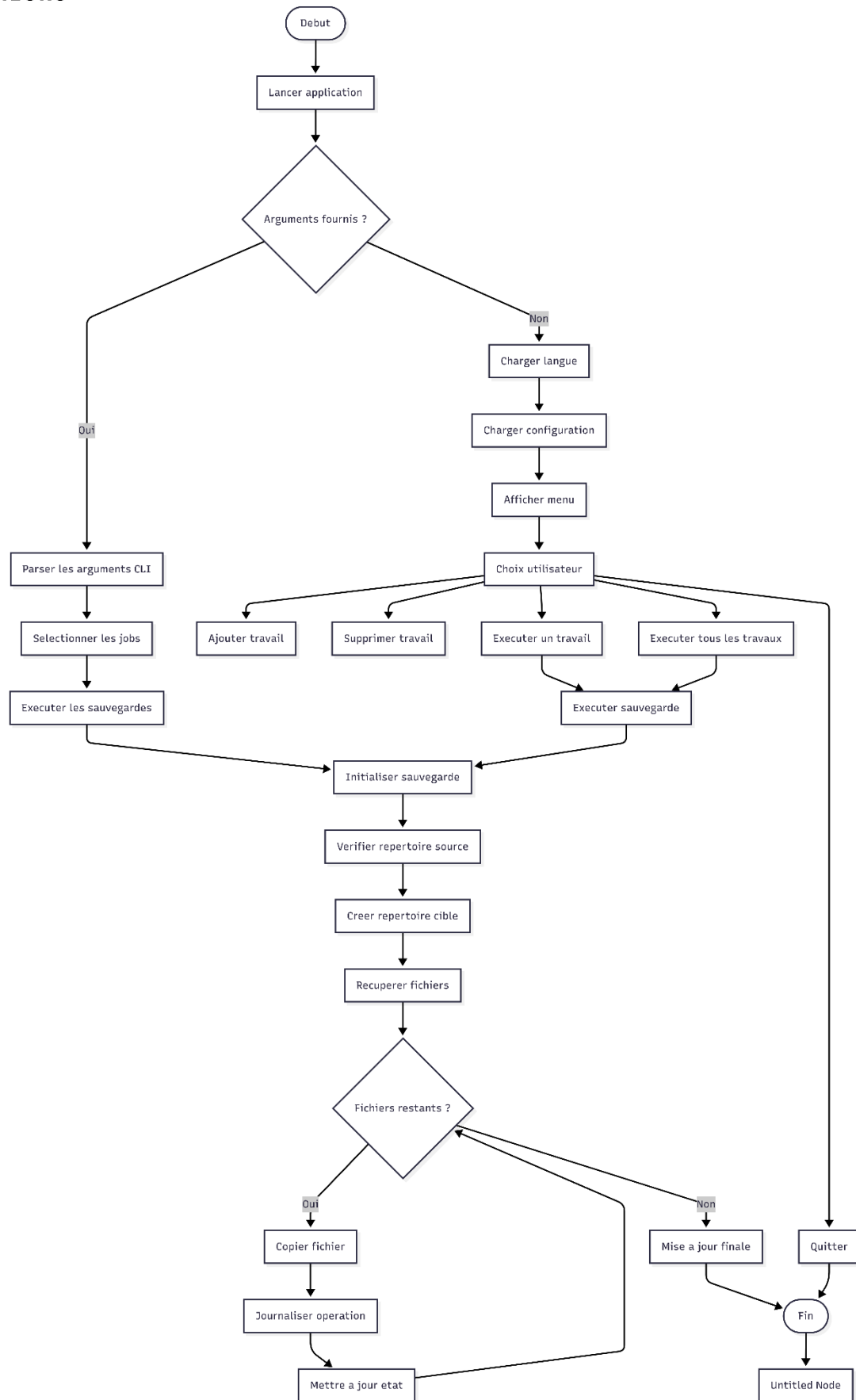


Figure 4 - Diagramme d'activité

Conclusion

Ce premier livrable du projet **EasySave (version 1.0)** marque une étape fondamentale dans la conception du logiciel de sauvegarde. Il a permis de passer d'une analyse des besoins exprimés par le cahier des charges à une modélisation structurée et cohérente du système à travers différents diagrammes UML.

L'étude des exigences fonctionnelles et non fonctionnelles a mis en évidence les principales attentes du système, notamment la gestion de travaux de sauvegarde limités en nombre, la possibilité d'exécution multiple, ainsi que la prise en charge de différents environnements de stockage. Ces contraintes ont guidé l'ensemble des choix de conception.

Sur le plan architectural, une organisation modulaire inspirée du modèle MVVM a été retenue afin de garantir une séparation claire des responsabilités entre les différentes couches du système. Cette approche favorise non seulement la lisibilité et la maintenabilité du code, mais également son évolution future. L'introduction de services spécialisés ainsi que d'une bibliothèque externe dédiée à la gestion des logs et de l'état (EasyLog.dll) permet également d'assurer une meilleure réutilisabilité des composants.

Les diagrammes UML réalisés constituent le cœur de ce livrable. Le diagramme de cas d'utilisation permet de définir les interactions entre l'utilisateur et le système, en mettant en évidence les différents modes d'exécution (interactif et ligne de commande). Le diagramme de classes offre une vision globale de l'architecture logicielle et des relations entre les entités principales. Le diagramme de séquence décrit de manière détaillée le déroulement d'une opération de sauvegarde, tandis que le diagramme d'activité représente le flux global d'exécution du programme.

L'ensemble de ces éléments permet de valider une compréhension claire et structurée du système avant son implémentation. Ils constituent une base essentielle pour le développement des versions suivantes, notamment la version 2.0 qui introduira une interface graphique et une architecture encore plus évoluée.

Enfin, ce travail met en évidence l'importance de la phase de conception dans un projet logiciel de taille réelle. Une modélisation rigoureuse en amont permet non seulement de réduire les erreurs lors du développement, mais également d'assurer une meilleure évolutivité et une maintenance facilitée du système sur le long terme.